

// Jivops

Guía completa — 30 lecciones

Arquitectura · Lenguaje y recursos · Componentes · State management
· Producción y CI/CD · Operación avanzada

Contenido

Módulo 1: Fundamentos y arquitectura

- Lección 01 — Qué es Pulumi y el enfoque de IaC con lenguajes reales
- Lección 02 — Arquitectura — engine, providers, state
- Lección 03 — Instalación y primer proyecto Pulumi
- Lección 04 — Pulumi vs Terraform — diferencias de enfoque
- Lección 05 — Lenguajes soportados — elegir el SDK adecuado

Módulo 2: Lenguaje y recursos

- Lección 06 — Recursos — declaración y ciclo de vida
- Lección 07 — Inputs y Outputs — el modelo asíncrono de Pulumi
- Lección 08 — Stacks — entornos múltiples de un mismo proyecto
- Lección 09 — Configuración y secretos por stack
- Lección 10 — Dependencias entre recursos — explícitas e implícitas

Módulo 3: Componentes y reutilización

- Lección 11 — Component Resources — encapsular infraestructura reutilizable
- Lección 12 — ComponentResource vs recursos nativos del provider
- Lección 13 — Pulumi Packages — publicar componentes reutilizables
- Lección 14 — Programación real — loops, condicionales, funciones
- Lección 15 — Testing de infraestructura (unit tests con el SDK)

Módulo 4: State management

- Lección 16 — Pulumi Service backend — state gestionado
- Lección 17 — Self-managed backends — S3, Azure Blob, GCS
- Lección 18 — Locking de state y concurrencia
- Lección 19 — Import de recursos existentes
- Lección 20 — Refresh y detección de drift

Módulo 5: Producción y CI/CD

- Lección 21 — Pulumi en pipelines de CI/CD
- Lección 22 — Pulumi Deployments — CI/CD gestionado nativo
- Lección 23 — Políticas con CrossGuard (Policy as Code)
- Lección 24 — Automation API — Pulumi programático
- Lección 25 — Preview y diff antes de aplicar cambios

Módulo 6: Operación avanzada

Lección 26 — Gestión de múltiples clouds con un solo proyecto

Lección 27 — RBAC y equipos en Pulumi Service

Lección 28 — Migración de Terraform a Pulumi

Lección 29 — Rendimiento — paralelismo y optimización de despliegues grandes

Lección 30 — Proyecto final: infraestructura completa con Pulumi lista para producción

Módulo 1: Fundamentos y arquitectura

Lección 01 — Qué es Pulumi y el enfoque de IaC con lenguajes reales

Usar construcciones nativas del lenguaje (loops, funciones) permite expresar lógica compleja de forma más natural que un lenguaje declarativo limitado.

Lección 02 — Arquitectura — engine, providers, state

El engine gestiona la lógica genérica de comparar estado deseado vs actual; cada provider traduce eso a llamadas reales de su API específica.

Lección 03 — Instalación y primer proyecto Pulumi

Una plantilla ya orientada a un proveedor ahorra configuración inicial repetitiva frente a empezar desde un proyecto completamente vacío.

```
pulumi new aws-typescript
```

Lección 04 — Pulumi vs Terraform — diferencias de enfoque

HCL, al ser limitado, tiende naturalmente a una estructura más uniforme entre proyectos que la libertad de un lenguaje de programación completo.

Lección 05 — Lenguajes soportados — elegir el SDK adecuado

Un equipo que ya escribe su app en TypeScript reduce el cambio de contexto mental usando el mismo lenguaje también para su infraestructura.

Módulo 2: Lenguaje y recursos

Lección 06 — Recursos — declaración y ciclo de vida

Si se elimina la declaración de un recurso del código, Pulumi lo elimina realmente del proveedor para que el estado real coincida con lo declarado.

Lección 07 — Inputs y Outputs — el modelo asíncrono de Pulumi

'`.apply()`' encadena una transformación para cuando el valor (como una IP) finalmente esté disponible tras crearse el recurso real.

Lección 08 — Stacks — entornos múltiples de un mismo proyecto

Cada Stack mantiene su propio state completamente separado, así que experimentar en 'dev' no afecta el state real de 'produccion'.

```
pulumi stack init produccion
```

Lección 09 — Configuración y secretos por stack

El archivo de configuración del Stack normalmente sí se versiona en Git; `--secret` cifra el valor antes de guardarlo ahí.

```
pulumi config set --secret dbPassword
```

Lección 10 — Dependencias entre recursos — explícitas e implícitas

Pulumi infiere la dependencia directamente del flujo real de datos en el código, más confiable que depender de declaración manual completa.

Módulo 3: Componentes y reutilización

Lección 11 — Component Resources — encapsular infraestructura reutilizable

Cada equipo instancia 'new MiVPC(...)' con parámetros simples, sin repetir la configuración detallada ya encapsulada en el componente.

Lección 12 — ComponentResource vs recursos nativos del provider

El beneficio principal viene de la reutilización; para un caso verdaderamente único, la abstracción adicional no se justifica con ningún beneficio real.

Lección 13 — Pulumi Packages — publicar componentes reutilizables

Publicar una nueva versión del paquete permite actualizar de forma controlada, en vez de localizar cada copia pegada manualmente en distintos proyectos.

Lección 14 — Programación real — loops, condicionales, funciones

Un cambio en la lógica del loop se aplica a los 10 recursos generados a la vez, en vez de actualizar manualmente 10 bloques casi idénticos.

Lección 15 — Testing de infraestructura (unit tests con el SDK)

Las pruebas con mocks se ejecutan en memoria casi instantáneamente, sin el costo real de aprovisionar recursos verdaderos en cada validación.

Módulo 4: State management

Lección 16 — Pulumi Service backend — state gestionado

El locking automático evita que dos ejecuciones simultáneas de 'pulumi up' generen una condición de carrera que corrompa el state.

Lección 17 — Self-managed backends — S3, Azure Blob, GCS

Una organización con requisitos de compliance específicos prefiere control directo sobre dónde reside su state, en vez de confiarlo a un servicio externo.

```
pulumi login s3://mi-bucket-state
```

Lección 18 — Locking de state y concurrencia

Sin locking, dos operaciones simultáneas podrían pisar sus cambios mutuamente, dejando el state en un estado inconsistente e impredecible.

Lección 19 — Import de recursos existentes

Importar adopta la gestión de un recurso ya funcional sin tocarlo, evitando la interrupción real que causaría eliminarlo y recrearlo desde cero.

```
pulumi import aws:s3/bucket:Bucket miBucket nombre-bucket-real
```

Lección 20 — Refresh y detección de drift

Si alguien cambia algo directamente en la consola del proveedor, refresh detecta esa discrepancia y actualiza el state local para reflejarla.

Módulo 5: Producción y CI/CD

Lección 21 — Pulumi en pipelines de CI/CD

Mostrar el preview en el pull request da al revisor visibilidad concreta del impacto real, en vez de aprobar el cambio solo confiando en el código fuente.

Lección 22 — Pulumi Deployments — CI/CD gestionado nativo

El equipo no necesita configurar y mantener un runner externo por separado, ya que la integración nativa está pensada específicamente para ese ciclo de vida.

Lección 23 — Políticas con CrossGuard (Policy as Code)

La política se aplica y bloquea automáticamente si no se cumple, garantizando consistencia sin depender de que cada desarrollador la recuerde aplicar.

```
pulumi up --policy-pack mi-politica
```

Lección 24 — Automation API — Pulumi programático

Una plataforma SaaS puede aprovisionar infraestructura personalizada por cliente directamente desde su propio código, sin intervención manual del CLI.

Lección 25 — Preview y diff antes de aplicar cambios

El preview da la última oportunidad de detectar una eliminación no intencionada de un recurso crítico antes de que el cambio real se ejecute.

```
pulumi preview --diff
```


Módulo 6: Operación avanzada

Lección 26 — Gestión de múltiples clouds con un solo proyecto

Tener AWS y Azure en el mismo programa permite expresar dependencias directas cross-cloud, mucho más difícil de coordinar entre proyectos separados.

Lección 27 — RBAC y equipos en Pulumi Service

Dar permisos de escritura solo a un grupo reducido sobre el Stack de producción reduce el riesgo de un cambio no autorizado.

Lección 28 — Migración de Terraform a Pulumi

Combinar 'pulumi convert' con 'pulumi import' traduce el código y adopta los recursos existentes sin destruir y recrear infraestructura real en uso.

```
pulumi convert --from terraform
```

Lección 29 — Rendimiento — paralelismo y optimización de despliegues grandes

Recursos sin dependencia entre sí se despliegan en paralelo automáticamente; solo los que dependen explícitamente esperan su turno secuencial.

```
pulumi up --parallel 20
```

Lección 30 — Proyecto final: infraestructura completa con Pulumi lista para producción

Se integra todo lo anterior: Stacks bien separados por entorno, Component Resources reutilizables, testing con mocks, políticas con CrossGuard, backend de state con locking adecuado e integración CI/CD con preview en pull requests — el nivel esperado de una infraestructura con Pulumi lista para producción real.